StreamDB

Guía de bases de datos del proyecto: conceptos de SQL y NoSQL, cómo usamos cada motor y cómo funcionan las consultas

MongoDB Redis Cassandra Neo4j PostgreSQL

Ingeniería de Datos II · UADE · TPO Etapa 2 · Plataforma de streaming
Todas las consultas de esta guía están tomadas del código del proyecto (`src/problemas/` , `src/setup/`).

Contenido

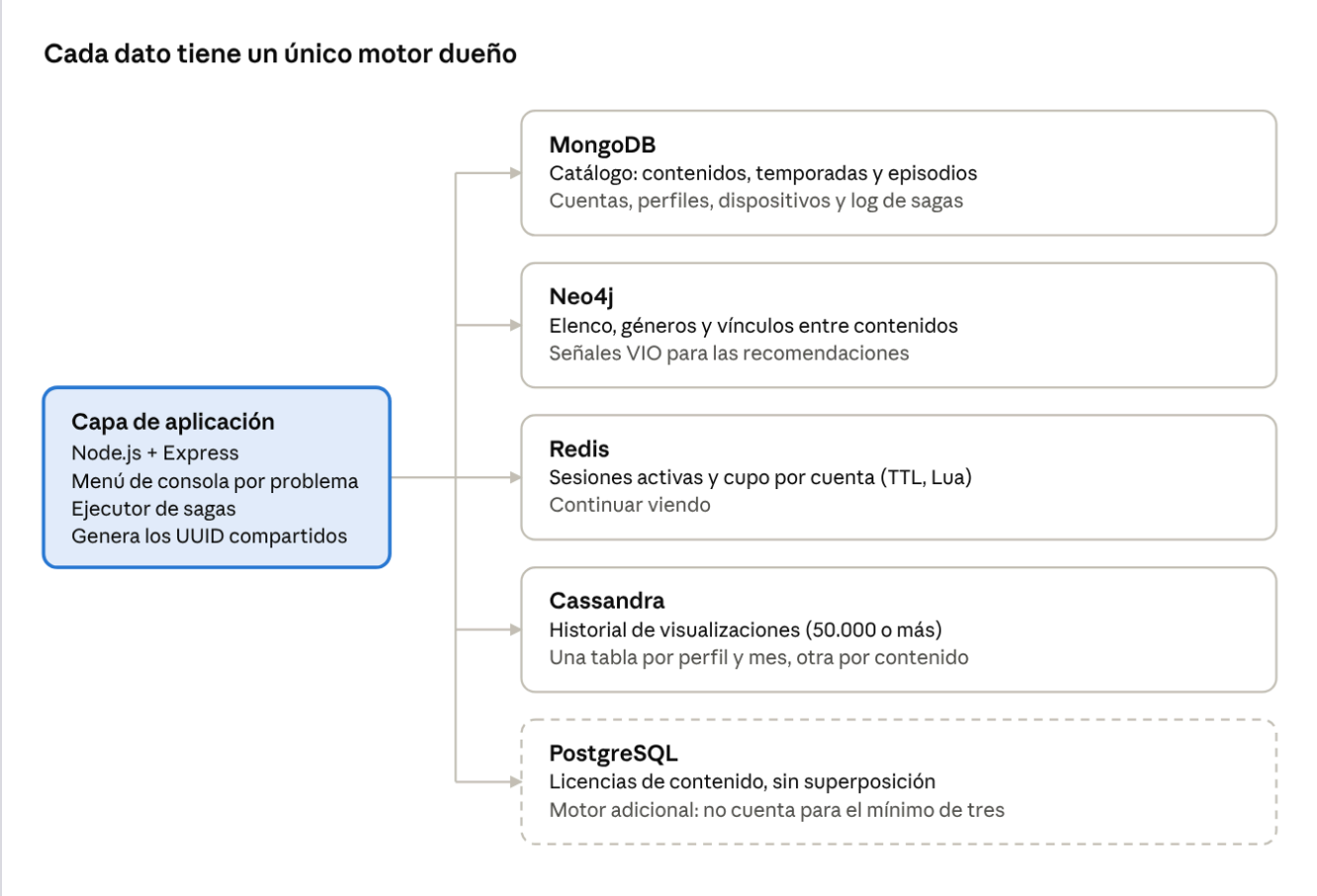
1. El proyecto en una página
2. Conceptos generales: SQL y NoSQL
3. MongoDB: documentos
4. Redis: clave-valor en memoria
5. Cassandra: columnas anchas
6. Neo4j: grafos
7. PostgreSQL: relacional
8. Coordinar varios motores: sagas
9. Los ocho ejercicios, paso a paso
10. "Continuar viendo" de punta a punta
11. Glosario

> **Cómo leer esta guía.** El capítulo 2 explica los conceptos de base (qué es una base relacional, qué cambia en NoSQL, ACID, CAP, consistencia). Los capítulos 3 a 7 toman un motor cada uno: primero el concepto, después cómo lo usamos y por último las consultas reales, explicadas línea por línea. El capítulo 8 muestra cómo se coordinan los motores cuando una operación los cruza, y el 9 recorre los ocho ejercicios del enunciado indicando qué hace cada base en cada paso.

1. El proyecto en una página

StreamDB es el backend de una plataforma de streaming: un catálogo de películas y series, cuentas con varios perfiles, reproducción con seguimiento del progreso, recomendaciones según lo que se vio, control de cuántos dispositivos pueden mirar a la vez y licencias que habilitan cada contenido.

El enunciado pide **persistencia políglota**: en vez de guardar todo en una sola base, cada dato vive en el motor que mejor se adapta a cómo se usa. Una capa de aplicación (Node.js con Express) recibe los pedidos y los reparte entre los motores.



Cada dato tiene un único motor dueño. La aplicación genera los identificadores y orquesta las operaciones que cruzan motores.

Motor	Modelo	Qué guarda	Por qué ahí
MongoDB	Documentos	Catálogo (contenido con temporadas y episodios), cuentas con perfiles, dispositivos, log de sagas	Una ficha se lee entera de una vez y tiene estructura anidada y variable (película o serie)
Redis	Clave-valor en memoria	Sesiones activas, cupo de pantallas por cuenta, "continuar viendo"	Lecturas y escrituras constantes, por clave exacta, con vencimiento automático y operaciones atómicas
Cassandra	Columnas anchas	Historial de visualizaciones (50.000 o más filas) y sesiones cerradas	Muchísimas escrituras, crece sin límite y siempre se consulta por perfil y mes, o por contenido
Neo4j	Grafo	Actores, géneros, vínculos entre contenidos (secuela, mismo universo, similar) y señales `VIO`	Recomendar es recorrer relaciones de varios saltos
PostgreSQL	Relacional	Licencias de contenido por región y período	La base tiene que impedir que dos licencias se superpongan: una restricción que ningún motor NoSQL ofrece

El identificador compartido

Un mismo contenido aparece en los cinco motores: su ficha en Mongo, su nodo en Neo4j, su "continuar viendo" en Redis, su historial en Cassandra y sus licencias en PostgreSQL. Lo que los une es un **UUID** que genera la aplicación (`nuevoId()` en `src/lib/ids.js`) antes de escribir en ninguna base. No hay claves foráneas entre motores: la coherencia la mantiene la aplicación.

Los ocho ejercicios del enunciado

Ejercicio	Qué hace	Motores
P1 Alta de serie	El admin carga una serie con temporadas, elenco y vínculos	Mongo, Neo4j
P2 Inicio de reproducción	Un perfil le da ▶; se controla el cupo de pantallas	Mongo, Redis, Cassandra
P3 Progreso	Mientras se mira, se guarda el segundo exacto	Redis, Cassandra
P4 Fin de episodio	Termina un episodio: historial, siguiente episodio y señal para recomendar	Cassandra, Redis, Neo4j
P5 Recomendaciones	"Porque viste X", recorriendo el grafo	Mongo, Redis, Neo4j
P6 Expiración y cierre	Sesiones que vencen solas, cierre manual o remoto	Redis, Cassandra
P7 Exploración	Filtrar el catálogo por género, actor y plataforma	Mongo, Neo4j, Redis
P8 Retiro y trazabilidad	Retirar un contenido cuya licencia venció; ver su rastro en los cinco motores	Los cinco

2. Conceptos generales: SQL y NoSQL

2.1 Bases relacionales (SQL)

Una base relacional guarda los datos en **tablas** con columnas fijas. Cada fila tiene una **clave primaria** que la identifica, y las tablas se relacionan con **claves foráneas**. El diseño se **normaliza**: cada dato se guarda una sola vez y, para leer información repartida en varias tablas, se las combina con `JOIN`. Se consulta con **SQL**, un lenguaje declarativo: se dice qué se quiere y el motor decide cómo buscarlo.

Su gran fortaleza son las **transacciones ACID**:

- **Atomicidad:** una transacción se hace entera o no se hace nada.
- **Consistencia:** la base pasa de un estado válido a otro; las restricciones (claves, `CHECK`, unicidad) nunca quedan violadas.
- **Aislamiento:** dos transacciones simultáneas no se ven a medias.
- **Durabilidad:** lo confirmado no se pierde aunque se corte la luz.

El costo: escalar a muchas máquinas es difícil (los `JOIN` y las transacciones necesitan los datos juntos), y el esquema rígido obliga a migraciones para cambiar la forma de los datos.

2.2 Qué cambia en NoSQL

"NoSQL" agrupa motores que abandonan el modelo de tablas para resolver mejor algún problema puntual: volumen enorme, velocidad extrema, datos con forma variable o relaciones complejas. Hay cuatro familias, y en el proyecto usamos una de cada una:

Familia	Unidad de dato	Fuerte en	En StreamDB
Documentos	Documento JSON con estructura anidada	Leer un objeto completo de una vez, esquema flexible	MongoDB
Clave-valor	Un valor bajo una clave	Velocidad (en memoria), contadores, colas, vencimientos	Redis
Columnas anchas	Filas agrupadas en particiones	Escrituras masivas, distribución en muchos nodos	Cassandra
Grafos	Nodos y relaciones con propiedades	Recorrer relaciones de varios saltos	Neo4j

Rasgos comunes de las bases NoSQL:

- **Se diseña por consulta.** En relacional se modela la realidad y después se consulta como se quiera. En NoSQL primero se definen las consultas y el modelo se arma para que cada una sea directa. Por eso en Cassandra tenemos dos tablas con los mismos datos.
- **Desnormalización.** Se copia información para no tener que juntarla al leer. Por ejemplo, la ficha de un contenido en Mongo guarda el nombre de la plataforma y de cada actor, y el nodo en Neo4j guarda el estado y la clasificación por edad.
- **Sin `JOIN` entre motores.** Combinar datos de dos bases es trabajo de la aplicación.

- **Escalado horizontal:** crecer agregando máquinas en vez de una máquina más grande.

2.3 Teorema CAP

Un sistema distribuido, ante una **partición** de red (P: algunos nodos no se pueden comunicar), tiene que elegir entre **consistencia** (C: todos ven el mismo dato) y **disponibilidad** (A: todos los pedidos reciben respuesta). No puede garantizar las dos a la vez.

- **Cassandra** es AP: sigue aceptando escrituras aunque algún nodo no responda, y las réplicas se ponen al día después (**consistencia eventual**). El nivel de consistencia se puede ajustar por consulta.
- **MongoDB** es CP por defecto: escribe en un nodo primario y, si se cae, no acepta escrituras hasta elegir otro.
- **Redis** y **Neo4j**, en un solo nodo como en el proyecto, son consistentes; con réplicas, Redis replica de forma asíncrona.

2.4 ACID y BASE

A la filosofía de muchas bases NoSQL se la resume como **BASE**: *Basically Available* (casi siempre responde), *Soft state* (el estado puede cambiar mientras las réplicas se sincronizan) y *Eventual consistency* (con el tiempo, todas coinciden). No es "peor" que ACID: es otra elección, que sacrifica garantías inmediatas a cambio de escala y disponibilidad.

En el proyecto se ven las dos: PostgreSQL protege las licencias con ACID y una restricción, Redis usa scripts atómicos, y cuando una operación cruza varios motores no hay transacción que los abarque, así que usamos **sagas** (capítulo 8).

2.5 Esquema

Las bases relacionales tienen **esquema en escritura**: la tabla define las columnas y no se puede guardar algo distinto. Muchas NoSQL tienen **esquema en lectura**: aceptan cualquier forma y la aplicación interpreta lo que lee. MongoDB está en el medio: es flexible, pero le pusimos validación con `$jsonSchema` para que rechace documentos mal formados (sección 3.3).

3. MongoDB: documentos MongoDB

3.1 Conceptos

- **Documento:** un objeto con forma de JSON (internamente BSON). Puede tener objetos y listas anidados. Equivale, de forma aproximada, a una fila con todas sus tablas hijas adentro.
- **Colección:** un conjunto de documentos, como una tabla sin columnas fijas.
- `_id` **:** la clave primaria de cada documento. Nosotros usamos el UUID compartido.
- **Embeber o referenciar:** si los datos se leen siempre juntos, se embeben (los episodios van dentro de la serie). Si tienen vida propia o son muchísimos, se referencian por id.
- **Índices:** como en SQL, evitan recorrer toda la colección. Pueden ser compuestos (varios campos), sobre campos de listas anidadas y parciales (solo para algunos documentos).
- **Pipeline de agregación:** una secuencia de etapas (`$match` , `$sort` , `$project` …) que transforman documentos, similar a encadenar `WHERE` , `ORDER BY` y `SELECT` .

3.2 Cómo lo usamos

Colección	Un documento es…	Embebido adentro
`contenidos`	Una película o serie	Plataforma, géneros, elenco (con el nombre del actor copiado), temporadas → episodios, datos del retiro
`cuentas`	Una cuenta	Hasta 5 perfiles, con su clasificación por edad
`dispositivos`	Un dispositivo	—
`sagas`	Una operación que cruza motores	La lista de pasos con su estado

Ejemplo de una ficha (resumida):

```
{
  "_id": "af5c457a-…",                 // el UUID compartido
  "titulo": "El faro del norte", "tipo": "serie", "estado": "publicado",
  "clasificacionEdad": 16, "anioEstreno": 1990,
  "plataforma": { "id": "…", "nombre": "Thames Drama" },
  "generos": ["Suspenso", "Historia", "Familiar"],
  "elenco": [ { "idActor": "…", "nombre": "Samuel Narváez Almonte", "personaje": "Pío Magaña
Pedroza" } ],
  "temporadas": [
    { "numero": 1, "episodios": [ { "id": "…", "numero": 1, "titulo": "La casa en llamas",
"duracionSeg": 3240 } ] }
  ]
}
```

3.3 Validación e índices (`src/setup/mongo.js`)

Cada colección se crea con un validador `$jsonSchema` . Si un documento no cumple, Mongo rechaza la escritura (código de error 121):

```
db.createCollection('contenidos', {
  validator: { $jsonSchema: {
    required: ['_id', 'titulo', 'tipo', 'estado', 'clasificacionEdad', 'plataforma'],
    properties: {
      tipo:   { enum: ['pelicula', 'serie'] },
      estado: { enum: ['borrador', 'publicado', 'retirado'] },
      clasificacionEdad: { bsonType: 'int', enum: [0, 7, 13, 16, 18] },
      …
  } } },
  validationLevel: 'strict', validationAction: 'error'
})
```

Índices de `contenidos`, pensados para la exploración (P7), que siempre filtra por `estado: 'publicado'` más otro criterio:

```
{ estado: 1, generos: 1 }            // "estado_genero"
{ estado: 1, 'elenco.idActor': 1 }   // "estado_actor": índice sobre un campo dentro de una lista
{ estado: 1, 'plataforma.id': 1 }    // "estado_plataforma"
{ 'temporadas.episodios.id': 1 }     // único y parcial: solo series (las películas no tienen episodios)
```

En `cuentas` hay un índice único por `email` y otro por `perfiles.id`, para encontrar la cuenta a partir de un perfil.

3.4 Consultas explicadas

Buscar la cuenta de un perfil, trayendo solo ese perfil (P2, P5, P7)

```
db.cuentas.findOne(
  { 'perfiles.id': idPerfil },                                  // usa el índice perfil_id
  { projection: { estado: 1, limiteSesiones: 1, 'perfiles.$': 1 } }
)
```

La notación con punto (`perfiles.id`) busca dentro de la lista embebida. La proyección `'perfiles.$'` es el **operador posicional**: de toda la lista de perfiles devuelve solo el que coincidió con el filtro. Así no viajan por la red los perfiles que no hacen falta.

Exploración con filtros y paginado (P7)

```
db.contenidos.aggregate([
  { $match: { estado: 'publicado', generos: 'Drama', clasificacionEdad: { $lte: 13 } } },
  { $sort: { anioEstreno: -1, _id: 1 } },
  { $facet: {
      total: [ { $count: 'n' } ],
      contenidos: [
        { $skip: (pagina - 1) * limite },
        { $limit: limite },
        { $project: {
            titulo: 1, tipo: 1, anioEstreno: 1, clasificacionEdad: 1, generos: 1,
            plataforma: '$plataforma.nombre',
            cantidadEpisodios: { $cond: [ { $eq: ['$tipo', 'serie'] },
              { $sum: { $map: { input: '$temporadas', as: 't', in: { $size: '$$t.episodios' }
} } },
              null ] }
        } }
      ]
  } }
])
```

1. `$match` filtra, como un `WHERE`. Se arma según los filtros recibidos: género, actor (`'elenco.idActor'`), plataforma y, si hay perfil, la edad (`$lte` = menor o igual). Al empezar por `estado`, puede usar los índices compuestos.
2. `$sort` ordena: lo más nuevo primero y, ante un empate, por `_id`, para que el paginado sea estable.
3. `$facet` corre dos sub-pipelines sobre el mismo resultado: uno cuenta el total (para saber cuántas páginas hay) y otro trae solo la página pedida (`$skip` + `$limit`). Las dos cosas llegan en una sola ida a la base.
4. `$project` elige los campos, como el `SELECT`. Además **calcula** la cantidad de episodios: `$map` recorre las temporadas y obtiene el tamaño (`$size`) de cada lista de episodios, y `$sum` los suma. Los episodios en sí no se envían.

Crear la ficha de forma idempotente (P1, paso `ficha`)

```
db.contenidos.updateOne({ _id: id }, { $setOnInsert: ficha }, { upsert: true })
```

Upsert = "actualizá si existe, insertá si no". `$setOnInsert` escribe la ficha solo si el documento se está creando. Si el paso se reintenta, no la pisa ni la duplica. Que un paso pueda repetirse sin efectos extra se llama **idempotencia**, y es clave en las sagas.

Retirar un contenido (P8, paso `retirar_ficha`)

```
db.contenidos.updateOne({ _id: id }, { $set: { estado: 'retirado', retiro: { idLicencia,
fecha, idSaga } } })
```

`$set` modifica solo esos campos; el resto del documento queda igual. Como todas las consultas de lectura filtran por `estado: 'publicado'`, el contenido deja de mostrarse en todos lados.

El log de sagas

Cada operación que cruza motores registra en `sagas` sus pasos y el estado de cada uno, con actualizaciones puntuales sobre un elemento de la lista:

```
db.sagas.updateOne({ _id: idSaga }, { $set: { 'pasos.2.estado': 'fallido', 'pasos.2.error':
'…', actualizadaEn: new Date() } })
```

`pasos.2` apunta al tercer elemento de la lista (se cuenta desde 0). Mongo es el lugar natural para este log: cada saga es un documento con forma variable y se lee completo.

4. Redis: clave-valor en memoria Redis

4.1 Conceptos

- **Clave-valor en memoria:** todo vive en RAM, así que leer y escribir lleva microsegundos. Se accede siempre por el nombre exacto de la clave; no hay consultas por contenido.
- **Tipos de valor:** no guarda solo texto, sino estructuras con sus propias operaciones:

Tipo	Qué es	Comandos
STRING	Un valor	`SET` , `GET` , `EXISTS`
HASH	Un objeto con campos	`HSET` , `HGETALL` , `HINCRBY`
SET	Conjunto sin repetidos	`SADD` , `SREM` , `SMEMBERS`
ZSET (sorted set)	Conjunto ordenado por un número (*score*)	`ZADD` , `ZRANGE` , `ZCARD` , `ZREMRANGEBYSCORE`
LIST	Lista, se usa como cola	`RPUSH` , `LPOP` , `LLEN`

- **TTL (time to live):** una clave puede vencer sola (`SET … EX 120` , `EXPIRE`). Es la base del control de sesiones.
- **Atomicidad:** Redis ejecuta los comandos de a uno (un solo hilo), así que cada comando es atómico. Para que **varios** comandos lo sean juntos se usa un **script Lua**: Redis lo ejecuta entero sin intercalar nada de otro cliente. Es el equivalente a una transacción.
- **Keyspace notifications:** Redis puede avisar por pub/sub cuando una clave vence. Lo usamos para cerrar sesiones expiradas.
- **Durabilidad:** al ser memoria, se puede perder información. Por eso lo importante tiene respaldo en Cassandra.

4.2 Las claves del proyecto

Clave	Tipo	Contenido
`cupo:{idCuenta}`	ZSET	Sesiones activas de la cuenta; score = última actividad. `ZCARD` = pantallas en uso
`sesion:{idSesion}`	HASH	Perfil, dispositivo, contenido, episodio, segundo, cantidad de reportes
`vida:{idSesion}`	STRING con TTL de 120 s	Mientras exista, la sesión está viva. Cada reporte la renueva
`disp:{idDispositivo}`	STRING con TTL	La sesión activa de ese dispositivo (una por dispositivo)
`viendo:{idContenido}`	ZSET	Sesiones que miran ese contenido ahora
`cv:{idPerfil}:{idContenido}`	HASH	"Continuar viendo": episodio, segundo y fecha
`continuar:{idPerfil}`	ZSET	Contenidos a medias del perfil, del más reciente al más viejo
`cv_perfiles:{idContenido}`	SET	Índice inverso: qué perfiles tienen ese contenido a medias
`retirado:{idContenido}`	STRING	Marca de que se retiró (P8)
`cierres_pendientes`	LIST	Cierres de sesión que no se pudieron registrar en Cassandra, para reintentar

> Con Redis el "modelo de datos" es la convención de nombres de las claves: `prefijo:id`. Los índices los mantenemos a mano (`continuar:`, `cv_perfiles:`), porque Redis no puede buscar "todas las claves cuyo valor contiene X".

4.3 Consultas explicadas

Reservar un lugar en el cupo de pantallas (P2, script `RESERVAR_CUPO`, simplificado)

```
local cupo, sesion, viendo, disp, vida = KEYS[1], KEYS[2], KEYS[3], KEYS[4], KEYS[6]
local idSesion, ahora, limite, ttl = ARGV[1], tonumber(ARGV[2]), tonumber(ARGV[3]),
tonumber(ARGV[4])

-- 1. sacar del cupo las sesiones que vencieron (su score es más viejo que ahora − TTL)
redis.call('ZREMRANGEBYSCORE', cupo, '-inf', ahora - ttl * 1000)

-- 2. ¿hay lugar?
if redis.call('ZCARD', cupo) >= limite then return 0 end       -- 0 = sin lugar → la API
responde 409

-- 3. ocupar el lugar y crear la sesión
redis.call('HSET', sesion, unpack(ARGV, 6))                     -- datos de la sesión
redis.call('ZADD', cupo, ahora, idSesion)                       -- entra al cupo
redis.call('SET', vida, '1', 'EX', ttl)                         -- vence en 120 s si no hay
reportes
redis.call('ZADD', viendo, ahora, idSesion)
redis.call('SET', disp, idSesion, 'EX', ttl)                    -- el dispositivo apunta a su
sesión
return 1
```

Por qué tiene que ser un script. Si la aplicación hiciera "leer `ZCARD`" y después "agregar con `ZADD`" en dos pedidos separados, dos dispositivos podrían leer "1 de 2" al mismo tiempo y entrar los dos, superando el límite. Dentro del script nadie puede meterse entre la lectura y la escritura. La prueba `npm run prueba:concurrencia` lo demuestra: con el script gana uno solo; con la versión ingenua terminan 6 sesiones sobre un límite de 2.

El script real tiene además un camino de **reutilización**: si el dispositivo ya tiene una sesión activa de la cuenta (por ejemplo, al pasar al episodio siguiente), reutiliza esa sesión en vez de ocupar otro lugar. El cupo limita **dispositivos** simultáneos, no reproducciones.

Reportar el progreso (P3, script `REPORTAR_PROGRESO`)

```
if redis.call('EXISTS', vida) == 0 then return -1 end           -- la sesión expiró → 410
if redis.call('EXISTS', retirado) == 1 then return -2 end       -- el contenido se retiró →
410
redis.call('HSET', cv, 'idEpisodio', unidad, 'segundo', segundo, 'actualizado', ahoraIso)
redis.call('ZADD', continuar, ahora, idContenido)               -- sube al primer lugar de
"continuar viendo"
redis.call('SADD', perfilesConCv, idPerfil)                     -- índice inverso para P8
redis.call('EXPIRE', vida, ttl)                                 -- heartbeat: la sesión sigue
viva
redis.call('ZADD', cupo, 'XX', ahora, idSesion)                 -- XX = solo actualiza si ya
estaba
return redis.call('HINCRBY', sesion, 'reportes', 1)             -- número de reporte (cada 6
→ Cassandra)
```

En una sola operación guarda el segundo exacto, actualiza los índices y renueva la vida de la sesión. Devuelve el número de reporte para que la aplicación decida si toca escribir un checkpoint en Cassandra.

Cerrar sesiones (P6, script `CERRAR`)

Recibe un modo: `cuenta` (cierre remoto de todas), `ids` (cierre manual) o `vencidas` (solo las que ya expiraron). Por cada sesión la saca del cupo, de `viendo:` y del dispositivo, borra sus claves y devuelve sus datos para registrar el cierre en Cassandra. En modo `cuenta` borra el cupo entero en la misma operación, así los lugares se liberan de inmediato.

Expiración automática

```
CONFIG SET notify-keyspace-events Ex          -- avisar cuando vence una clave
SUBSCRIBE __keyevent@0__:expired               -- llega "vida:{idSesion}" al vencer
```

Cuando vence `vida:{id}`, la aplicación cierra esa sesión con motivo `expirada`. Como las notificaciones de Redis se pueden perder (si la API estaba caída), hay dos garantías más: un barrido cada 30 segundos (`SCAN` sobre `sesion:*`) y la limpieza del propio `RESERVAR_CUPO` (paso 1).

5. Cassandra: columnas anchas Cassandra

5.1 Conceptos

- **Distribuida desde el diseño:** los datos se reparten entre muchos nodos, sin un nodo principal. Cualquier nodo acepta escrituras, y cada dato se replica en varios (el *replication factor*; en el proyecto es 1 porque hay un solo nodo).
- **Keyspace:** el equivalente a una base de datos. Define la replicación.
- **Clave primaria = clave de partición + claves de clustering:**
  - La **clave de partición** decide en qué nodo vive la fila (se le aplica un hash). Todas las filas con la misma clave de partición están juntas: eso es una **partición**.
  - Las **claves de clustering** ordenan las filas **dentro** de la partición, en el disco.
- **Se diseña por consulta:** una consulta eficiente lee **una** partición. Por eso se hace una tabla por cada consulta, aunque eso duplique datos.
- **Sin `JOIN` y con `WHERE` limitado:** hay que dar la clave de partición completa. Filtrar por otra columna obliga a `ALLOW FILTERING`, que recorre todo el cluster. En el proyecto no lo usamos nunca.
- **Escrituras baratas:** se agregan a un log en memoria y se vuelcan a disco de forma secuencial. Ideal para algo que crece sin parar, como un historial.
- **`INSERT` y `UPDATE` son *upsert*:** escribir dos veces la misma clave no duplica, sobrescribe. Si gana el último, repetir es seguro.

5.2 Las tablas (`src/setup/cassandra.js`)

```
CREATE KEYSPACE IF NOT EXISTS streamdb
  WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

-- Consulta: "historial de un perfil en un período" (P3)
CREATE TABLE historial_por_perfil (
  id_perfil uuid, mes text, fecha timestamp, id_visualizacion uuid,
  id_contenido uuid, id_episodio uuid, id_sesion uuid,
  fecha_fin timestamp, posicion_seg int, finalizado boolean,
  PRIMARY KEY ((id_perfil, mes), fecha, id_visualizacion)
) WITH CLUSTERING ORDER BY (fecha DESC, id_visualizacion ASC);

-- Consulta: "historial de un contenido" (trazabilidad, P8)
CREATE TABLE historial_por_contenido (
  id_contenido uuid, fecha timestamp, id_visualizacion uuid, id_perfil uuid, …
  PRIMARY KEY ((id_contenido), fecha, id_visualizacion)
) WITH CLUSTERING ORDER BY (fecha DESC, id_visualizacion ASC);

-- Consulta: "sesiones cerradas de una cuenta" (P6)
CREATE TABLE sesiones_por_cuenta (
  id_cuenta uuid, fecha_inicio timestamp, id_sesion uuid,
  id_perfil uuid, id_dispositivo uuid, fecha_fin timestamp, motivo_cierre text,
  PRIMARY KEY ((id_cuenta), fecha_inicio, id_sesion)
) WITH CLUSTERING ORDER BY (fecha_inicio DESC, id_sesion ASC);
```

Cómo leer `PRIMARY KEY ((id_perfil, mes), fecha, id_visualizacion)`:

- `(id_perfil, mes)`, con doble paréntesis, es la **clave de partición compuesta**: cada perfil tiene una partición por mes. Si fuera solo `id_perfil`, la partición de un perfil crecería para siempre. Cortar por mes la mantiene acotada (patrón *bucketing*).
- `fecha DESC` ordena las visualizaciones de la más nueva a la más vieja: leer "las últimas" es leer el comienzo de la partición, sin ordenar nada.
- `id_visualizacion` desempata dos visualizaciones que empiecen en el mismo instante y hace única a cada fila.

> **Dos tablas con los mismos datos.** Cada visualización se escribe en `historial_por_perfil` y en `historial_por_contenido`. En relacional sería redundancia a evitar. En Cassandra es lo correcto: cada tabla responde una consulta leyendo una sola partición.

5.3 Consultas explicadas

Crear la fila de una visualización (P2, paso `registrar_inicio`)

```
INSERT INTO historial_por_perfil
  (id_perfil, mes, fecha, id_visualizacion, id_contenido, id_episodio, id_sesion, fecha_fin,
posicion_seg, finalizado)
  VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?);
-- y lo mismo en historial_por_contenido
```

Los `?` son parámetros: la consulta se **prepara** una vez y después solo viajan los valores. Es más rápido y evita inyección. El `mes` se calcula de la fecha (`'2026-10'`).

Checkpoint del progreso (P3)

```
UPDATE historial_por_perfil SET posicion_seg = ?, fecha_fin = ?
  WHERE id_perfil = ? AND mes = ? AND fecha = ? AND id_visualizacion = ?;
```

El `WHERE` da la clave primaria **completa**: apunta a una fila exacta. Como es un upsert con valor absoluto (el segundo, no "sumá 30"), si un checkpoint falla, el siguiente lo corrige. Se escribe cada 6 reportes y al pausar o cerrar.

Historial de un perfil (P3, "Mi actividad")

```
SELECT fecha, id_visualizacion, id_contenido, id_episodio, id_sesion, fecha_fin, posicion_seg,
finalizado
  FROM historial_por_perfil WHERE id_perfil = ? AND mes = ? LIMIT ?;
```

Para un período de varios meses, la aplicación hace **una consulta por mes**: cada una lee exactamente una partición. Es el precio de la partición por mes, y por eso no hace falta `ALLOW FILTERING`.

Leer antes de compensar (P4)

```
SELECT posicion_seg, fecha_fin, finalizado FROM historial_por_perfil
  WHERE id_perfil = ? AND mes = ? AND fecha = ? AND id_visualizacion = ?;
```

Antes de marcar la visualización como terminada, se guardan los valores anteriores: si la saga falla más adelante, la compensación los restaura.

Trazabilidad de un contenido (P8)

```
SELECT … FROM historial_por_contenido WHERE id_contenido = ? LIMIT ?;
SELECT count(*) AS total FROM historial_por_contenido WHERE id_contenido = ?;
```

Por eso existe la segunda tabla: "quién vio este contenido" sería imposible de responder bien con `historial_por_perfil`, que está partida por perfil. Al retirar un contenido, su historial **se conserva**.

6. Neo4j: grafos Neo4j

6.1 Conceptos

- **Grafo de propiedades: nodos** (entidades) unidos por **relaciones** (aristas con dirección y tipo). Ambos pueden tener propiedades.
- **Etiquetas:** clasifican los nodos (`:Contenido` , `:Actor` , `:Genero` , `:Perfil`).
- **Adyacencia sin índice:** cada nodo apunta directamente a sus relaciones. Ir de un nodo a sus vecinos cuesta lo mismo aunque el grafo tenga millones de nodos. En relacional, cada salto sería un `JOIN` , y tres o cuatro saltos se vuelven caros.
- **Cypher:** el lenguaje de consultas, basado en dibujar el patrón buscado con "arte ASCII": `(nodo)-[:RELACION]->(otro)` .

6.2 El grafo del proyecto

```
(:Actor)-[:ACTUA_EN {personaje}]->(:Contenido)
(:Contenido)-[:DE_GENERO]->(:Genero)
(:Contenido)-[:RELACIONADO {tipo: 'secuela' | 'mismo_universo' | 'similar'}]->(:Contenido)
(:Perfil)-[:VIO {idVisualizacion, fecha}]->(:Contenido)
```

El nodo `:Contenido` copia de Mongo el título, el estado y la clasificación por edad (**desnormalización**), para poder filtrar durante el recorrido sin consultar otra base. `(A)-[:RELACIONADO {tipo:'secuela'}]->(B)` significa "A es secuela de B". Una precuela se guarda como la secuela al revés.

Restricciones (`src/setup/neo4j.js`):

```
CREATE CONSTRAINT contenido_id IF NOT EXISTS FOR (c:Contenido) REQUIRE c.id IS UNIQUE;
CREATE CONSTRAINT vio_visualizacion IF NOT EXISTS FOR ()-[v:VIO]-() REQUIRE v.idVisualizacion
IS UNIQUE;
CREATE INDEX contenido_estado IF NOT EXISTS FOR (c:Contenido) ON (c.estado);
```

La unicidad también crea un índice para encontrar un nodo por `id` . La de `VIO` garantiza una sola señal por visualización.

6.3 Consultas explicadas

Cláusulas básicas

Cláusula	Hace	Análoga SQL
`MATCH`	Busca el patrón	`FROM … JOIN`
`OPTIONAL MATCH`	Igual, pero si no hay coincidencia sigue con `null`	`LEFT JOIN`
`WHERE`	Filtra	`WHERE`
`CREATE` / `MERGE`	Crea / crea solo si no existe	`INSERT` / upsert
`SET` , `REMOVE` , `DELETE`	Modifica o borra	`UPDATE` , `DELETE`
`WITH`	Pasa resultados a la parte siguiente (encadena)	Subconsulta
`UNWIND`	Convierte una lista en filas	—
`collect()` , `count()`	Agregan	`array_agg` , `COUNT`

Crear el nodo de una serie con sus géneros (P1, paso `nodo`)

```
MERGE (c:Contenido {id: $id})
SET c.titulo = $titulo, c.tipo = 'serie', c.estado = 'borrador', c.clasificacionEdad = $edad
WITH c UNWIND $generos AS nombre
MATCH (g:Genero {nombre: nombre})
MERGE (c)-[:DE_GENERO]->(g)
```

`MERGE` busca el nodo y lo crea solo si no existe (idempotente). `UNWIND` convierte la lista de géneros en una fila por género. Por cada una, `MATCH` encuentra el nodo del género y `MERGE` crea la relación si falta. Nace como `borrador` : no aparece en recomendaciones hasta el paso `publicar` .

Registrar que un perfil vio algo (P4, paso `senal_vio`)

```
MERGE (p:Perfil {id: $idPerfil})
WITH p MATCH (c:Contenido {id: $idContenido})
MERGE (p)-[r:VIO {idVisualizacion: $idVisualizacion}]->(c)
ON CREATE SET r.fecha = datetime($fecha)
```

El `MERGE` sobre la relación usa `idVisualizacion` : si el paso se repite, no se crea una segunda señal. `ON CREATE SET` pone la fecha solo la primera vez. La compensación borra exactamente esa relación: `MATCH ()-[r:VIO {idVisualizacion: $id}]->() DELETE r` .

Recomendaciones (P5, consulta `RECORRER` , simplificada)

Es la consulta más importante del grafo. Parte de unas **semillas** (lo que el perfil está viendo, tomado de Redis, y lo último que terminó, tomado de las relaciones `VIO`) y busca contenidos conectados por tres caminos:

```
UNWIND $semillas AS sem
MATCH (s:Contenido {id: sem.id})
CALL (s) {
  MATCH (s)-[r:RELACIONADO]-(c:Contenido)                          // 1. vínculo directo
  RETURN c, <secuela | precuela | mismo_universo | similar> AS motivo, null AS via
  UNION
  MATCH (s)<-[:ACTUA_EN]-(a:Actor)-[:ACTUA_EN]->(c:Contenido)       // 2. comparten un actor
  RETURN c, 'elenco' AS motivo, a.nombre + ' ' + a.apellido AS via
  UNION
  MATCH (s)-[:DE_GENERO]->(g:Genero)<-[:DE_GENERO]-(c:Contenido)    // 3. comparten un género
  RETURN c, 'genero' AS motivo, g.nombre AS via
}
WITH sem, s, c, motivo, via
WHERE c.id <> s.id AND NOT c.id IN $excluir                        // no lo que está viendo
  AND c.estado = 'publicado' AND c.clasificacionEdad <= $edad       // publicado y apto
  AND NOT EXISTS { MATCH (:Perfil {id: $idPerfil})-[:VIO]->(c) }    // no lo ya visto
WITH c, collect({ motivo: motivo, via: via, semilla: s.titulo, peso: $pesos[motivo] *
sem.factor }) AS caminos
WITH c, caminos, reduce(total = 0.0, x IN caminos | total + x.peso) AS puntaje
ORDER BY puntaje DESC, c.id
LIMIT toInteger($cantidad)
RETURN c.id AS id, puntaje, caminos
```

1. `UNWIND $semillas` procesa cada semilla. Cada una trae un `factor`: las más viejas pesan un poco menos.
2. `CALL (s) { … UNION … }` es una subconsulta que, para cada semilla, junta los candidatos de los tres caminos. El segundo patrón recorre **dos saltos** (semilla ← actor → candidato) y el tercero, otros dos (semilla → género ← candidato). En SQL serían varios `JOIN` por cada camino.
3. `WHERE` descarta lo que no sirve. Estado y edad están en el nodo, así que se filtra sin salir del grafo. `NOT EXISTS { … }` es una subconsulta de patrón: "que no exista una relación `VIO` de este perfil a este candidato".
4. `collect` agrupa todos los caminos que llegan al mismo candidato, y `reduce` suma sus pesos: `secuela 5`, `mismo_universo 4`, `precuela 3`, `similar 3`, `elenco 2`, `genero 1`. Un contenido al que se llega por varios caminos suma más.
5. Se ordena por puntaje y se cortan los mejores. El camino de más peso da el texto "*Porque viste **X**: es su secuela*".

Si el perfil no tiene historial, otra consulta devuelve lo más visto: cuenta las relaciones `VIO` que llegan a cada contenido (`count(v)`). Al final, las fichas se piden a Mongo, que es la fuente de verdad, y se revalida que sigan publicadas y aptas.

Vínculos de cada resultado de la exploración (P7)

```
MATCH (c:Contenido) WHERE c.id IN $ids
OPTIONAL MATCH (c)-[r:RELACIONADO]-(o:Contenido)
WHERE o.estado = 'publicado' AND ($edad IS NULL OR o.clasificacionEdad <= $edad)
RETURN c.id AS id, collect(CASE WHEN o IS NULL THEN null ELSE { id: o.id, titulo: o.titulo,
relacion: … } END) AS vinculados
```

Una sola consulta para toda la página de resultados (`IN $ids`), no una por contenido. `OPTIONAL MATCH` mantiene los contenidos sin vínculos, que devuelven una lista vacía. La relación no tiene dirección en el patrón (`-[r]-`), así encuentra vínculos en los dos sentidos.

7. PostgreSQL: relacional PostgreSQL

7.1 Por qué una base relacional para las licencias

Una **licencia** es el permiso para mostrar un contenido en una región durante un período. La regla de negocio es: **un contenido no puede tener dos licencias superpuestas en la misma región**. Con una base NoSQL habría que verificarlo en la aplicación (leer las licencias existentes y después insertar), lo que deja una ventana para que dos inserciones simultáneas pasen las dos. PostgreSQL lo resuelve con una **restricción declarativa**: es la base la que lo garantiza, también ante pedidos concurrentes.

Es un motor adicional: el enunciado pide al menos tres motores NoSQL, y PostgreSQL se suma solo para esta necesidad.

7.2 La tabla (`src/setup/postgres.js`)

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE IF NOT EXISTS licencia (
  id_licencia   UUID PRIMARY KEY,
  id_contenido  UUID NOT NULL,          -- mismo id que en los otros motores (sin FK: el
contenido vive en Mongo)
  id_plataforma UUID NOT NULL,
  region        CHAR(2) NOT NULL,       -- código de país: 'AR', 'UY', …
  vigencia      DATERANGE NOT NULL,     -- [desde, hasta)
  estado        TEXT NOT NULL DEFAULT 'vigente'
                CHECK (estado IN ('vigente', 'vencida', 'rescindida')),
  CONSTRAINT licencia_sin_superposicion
    EXCLUDE USING gist (id_contenido WITH =, region WITH =, vigencia WITH &&)
    WHERE (estado <> 'rescindida')
);
CREATE INDEX licencia_estado_fin ON licencia (estado, upper(vigencia));
CREATE INDEX licencia_contenido  ON licencia (id_contenido);
```

- `DATERANGE` es un tipo de PostgreSQL que guarda un intervalo de fechas en un solo valor. `[2023-03-01, 2027-03-01)` incluye el inicio y excluye el fin: dos licencias consecutivas no se pisan.
- `CHECK` limita los valores posibles del estado.
- `EXCLUDE` es una restricción más general que `UNIQUE` : prohíbe que existan dos filas para las que **todas** las comparaciones den verdadero. Acá: mismo contenido (`=`), misma región (`=`) y períodos que se superponen (`&&`). Si se intenta insertar una licencia así, la base la rechaza.
- `WHERE (estado <> 'rescindida')` : las licencias rescindidas no cuentan para la regla.
- `USING gist` y `btree_gist` : la restricción necesita un índice que entienda tanto la igualdad de UUID y texto como la superposición de rangos. El índice GiST maneja rangos, y la extensión `btree_gist` le agrega los tipos comunes.
- El índice sobre `(estado, upper(vigencia))` es un **índice sobre una expresión**: acelera "licencias vigentes cuyo fin ya pasó".

7.3 Consultas explicadas (P8)

¿Qué contenidos tienen licencias vigentes que ya terminaron?

```
SELECT DISTINCT id_contenido FROM licencia
WHERE estado = 'vigente' AND upper(vigencia) <= $1::date;
```

`upper()` devuelve el fin del rango y `lower()`, el inicio. `$1::date` es un parámetro convertido a fecha. El "procesar vencimientos" arranca por acá.

Licencias vigentes de un contenido y si alguna cubre la fecha

```
SELECT id_licencia, region, lower(vigencia) AS desde, upper(vigencia) AS hasta,
       vigencia @> $2::date AS cubre_fecha
FROM licencia WHERE id_contenido = $1 AND estado = 'vigente';
```

`@>` es el operador "contiene": ¿la fecha cae dentro del rango? Si alguna licencia todavía cubre la fecha (otra región o una renovación), el contenido no se retira: solo se marcan como vencidas las que terminaron.

Vencer licencias (último paso de la saga de retiro)

```
UPDATE licencia SET estado = 'vencida'
WHERE id_licencia = ANY($1::uuid[]) AND estado = 'vigente'
RETURNING id_licencia;
```

`ANY($1::uuid[])` recibe una lista de ids en un solo parámetro. La condición `AND estado = 'vigente'` hace la operación idempotente: repetirla no cambia nada. `RETURNING` devuelve qué filas cambió, para poder compensar (volverlas a `vigente`).

Carga masiva del seed

```
INSERT INTO licencia (id_licencia, id_contenido, id_plataforma, region, vigencia)
SELECT * FROM UNNEST($1::uuid[], $2::uuid[], $3::uuid[], $4::char(2)[], $5::daterange[]);
```

`UNNEST` convierte varias listas paralelas en filas: se insertan 500 licencias con un solo viaje a la base, en vez de 500 `INSERT`.

8. Coordinar varios motores: sagas

Cuando una operación escribe en dos o más motores (por ejemplo, P1 en Mongo y Neo4j), no existe una transacción que los abarque a todos. Si falla la escritura en Neo4j después de haber escrito en Mongo, Mongo quedaría con datos huérfanos. La solución es el **patrón saga**:

1. La operación se divide en **pasos**, cada uno en un solo motor.
2. Cada paso tiene una **compensación**: la acción que lo deshace.
3. Se ejecutan en orden. Si uno falla, se ejecutan las compensaciones de los pasos ya hechos, **en orden inverso**.

No es atomicidad real: durante un instante otros pueden ver un estado intermedio. Por eso los pasos están ordenados para que ese estado sea inofensivo. Por ejemplo, P1 crea todo como `borrador` y recién el último paso lo publica, y P8 vence la licencia al final, así que, si algo falla antes, el próximo proceso lo reintenta.

8.1 El ejecutor (`src/lib/saga.js`)

1. **Antes de escribir nada**, inserta en Mongo un documento en `sagas` con todos los pasos en `pendiente`.
2. Ejecuta cada paso y lo marca `hecho`. Lo que devuelve un paso (por ejemplo, el valor anterior) se guarda para su compensación.
3. Si un paso falla: lo marca `fallido`, compensa los anteriores en orden inverso marcándolos `compensado`, y la saga termina `compensada`.
4. Si además falla una compensación, la saga queda `fallida`: es el único caso que requiere revisión manual, y el log dice exactamente qué quedó sin deshacer.

Para demostrarlo, cada endpoint acepta `?fallarEn=<paso>`, que provoca una falla justo en ese paso. En la app de demo es la pestaña **Admin → Fallas**.

8.2 Idempotencia

Los pasos y las compensaciones se pueden repetir sin efectos extra: `upsert` y `$setOnInsert` en Mongo, `MERGE` en Neo4j, `UPDATE … WHERE estado = 'vigente'` en PostgreSQL, scripts Lua que borran "si existe" en Redis, upserts por clave en Cassandra. Esto importa porque, ante un error de red, no siempre se sabe si un paso se aplicó, y reintentarlo tiene que ser seguro.

8.3 Las sagas del proyecto

Saga	Pasos, en orden
P1 `alta_serie`	`ficha` (Mongo) → `episodios` (Mongo) → `nodo` (Neo4j) → `elenco` (Neo4j) → `vinculos` (Neo4j) → `publicar` (Neo4j y Mongo)
P2 `inicio_reproduccion`	`reservar_cupo` (Redis) → `registrar_inicio` (Cassandra)
P4 `fin_episodio`	`marcar_finalizado` (Cassandra) → `continuar_viendo` (Redis) → `senal_vio` (Neo4j)
P8 `retiro_contenido`	`continuar_viendo` (Redis) → `marcar_nodo` (Neo4j) → `retirar_ficha` (Mongo) → `vencer_licencias` (PostgreSQL)

> **Validar antes de empezar.** Las sagas validan todo antes del primer paso: que los géneros, actores y vínculos existan (P1), que el perfil sea apto para el contenido (P2), que no quede una licencia vigente (P8). Si algo no cumple, se responde el error sin escribir nada y no hay nada que compensar.

9. Los ocho ejercicios, paso a paso

P1 · Alta de serie Mongo Neo4j

1. **Valida** la entrada y que existan la plataforma (Mongo), los géneros y los actores (Neo4j) y los contenidos a vincular, que además no pueden estar retirados.
2. **Saga:** la ficha se crea en Mongo como `borrador`, se agregan los episodios, se crean el nodo con sus géneros, el elenco (`ACTUA_EN`) y los vínculos (`RELACIONADO`) en Neo4j, y al final se publica en los dos.
3. Hasta el último paso no aparece en ningún lado, porque todo filtra por `publicado`.

P2 · Inicio de reproducción Mongo Redis Cassandra

1. **Mongo:** busca la cuenta del perfil (estado y `limiteSesiones`), el dispositivo y el contenido (publicado y apto para la edad). Si no es apto, responde 403.
2. **Redis:** lee el "continuar viendo" para saber desde qué episodio y segundo retomar.
3. **Saga:** `reservar_cupo` (script atómico; sin lugar → **409**) y `registrar_inicio` (fila nueva en las dos tablas de historial de Cassandra). Si falla Cassandra, se libera el lugar en Redis.

P3 · Progreso Redis Cassandra

1. Cada pocos segundos, el reproductor informa el segundo actual. **Redis** lo guarda con `REPORTAR_PROGRESO` y renueva la sesión.
2. Cada 6 reportes, y al pausar o cerrar, se hace un **checkpoint** en Cassandra (`UPDATE` sobre la fila de la visualización).
3. Si la sesión expiró o el contenido se retiró, responde 410 y no escribe nada.

No es una saga: si el checkpoint falla, Redis ya tiene el dato y el próximo checkpoint, que es un upsert del valor absoluto, lo corrige.

P4 · Fin de episodio Cassandra Redis Neo4j

1. **Mongo** (lectura): la duración y cuál es el episodio siguiente.
2. **Saga:** Cassandra marca la visualización como `finalizado` (guardando los valores previos). Redis mueve el "continuar viendo" al siguiente episodio en el segundo 0, o lo saca si no hay. Neo4j crea la señal `VIO`.
3. Repetir el pedido devuelve `yaRegistrado` sin escribir de nuevo.

P5 · Recomendaciones Mongo Redis Neo4j

1. **Mongo:** la clasificación por edad del perfil.
2. **Redis:** lo que está viendo (`continuar:{perfil}`). **Neo4j:** lo último que terminó (`VIO`). Las dos cosas son semillas.
3. **Neo4j:** la consulta `RECORRER` (sección 6.3) puntúa candidatos por secuela, universo, similitud, elenco y género.
4. **Mongo:** trae las fichas de los candidatos y revalida que estén publicados y sean aptos.

P6 · Expiración y cierre de sesiones Redis Cassandra

- **Expiración:** si pasan 120 s sin reportes, vence `vida:{id}`. Redis avisa, la aplicación cierra la sesión (`CERRAR` en modo `vencidas`) y libera el lugar. Hay un barrido de respaldo cada 30 s.
- **Cierre remoto:** "cerrar sesión en todos lados" vacía el cupo de la cuenta en una sola operación.
- **Cierre manual:** el dispositivo cierra su sesión.
- En los tres casos se inserta una fila en `sesiones_por_cuenta` (Cassandra) con el motivo: `expirada`, `cierre_remoto` o `manual`. Si Cassandra falla, el cierre en Redis ya está hecho y el registro queda en la lista `cierres_pendientes` para reintentarlo.

P7 · Exploración Mongo Neo4j Redis

1. **Mongo:** la agregación de la sección 3.4, con los filtros, la edad del perfil y el paginado.
2. **En paralelo:** Neo4j trae los vínculos de los resultados de la página, y Redis, el "continuar viendo" del perfil en cada uno.

P8 · Retiro y trazabilidad PostgreSQL Redis Neo4j Mongo

1. **PostgreSQL:** si al contenido le queda una licencia que cubre la fecha, no se puede retirar (409).
2. **Saga:**
   - Redis borra el "continuar viendo" de todos los perfiles, usando `cv_perfiles:`, y marca `retirado:`.
   - Neo4j marca el nodo como retirado.
   - Mongo marca la ficha como retirada.
   - PostgreSQL vence las licencias.
3. **Cassandra no se toca:** el historial se conserva.
4. **Trazabilidad:** una sola consulta lanza en paralelo la lectura de los cinco motores, todos con el mismo id: la ficha (Mongo), los vínculos y señales (Neo4j), quién lo mira ahora (Redis), el historial (Cassandra) y las licencias (PostgreSQL). Si un motor no responde, se informa cuál y se devuelve lo del resto.

10. "Continuar viendo" de punta a punta

Es un buen ejemplo de cómo un mismo dato pasa por varios motores, cada uno con un rol.

Motor	Rol
Redis	**Fuente rápida.** `cv:{perfil}:{contenido}` (episodio y segundo), `continuar:{perfil}` (orden por actividad) y `cv_perfiles:{contenido}` (índice inverso)
Cassandra	**Respaldo durable.** El checkpoint del segundo en la fila de la visualización
MongoDB	**Fuente de verdad del catálogo.** Los títulos, y solo se muestra lo publicado
Neo4j	No guarda el "continuar viendo". Recibe la señal `VIO` cuando se termina algo, y P5 usa el "continuar viendo" como semilla

1. ▶ **(P2):** se lee el `cv` para retomar desde el episodio y segundo guardados.
2. **Mientras se mira (P3):** cada reporte actualiza el `cv` y lo sube en `continuar:`. Cada 6 reportes, checkpoint en Cassandra.
3. **Al terminar (P4):** el `cv` pasa al episodio siguiente desde 0, o sale de la lista si no hay siguiente. En Neo4j queda la señal `VIO`.
4. **Al retirar (P8):** se borra el `cv` de todos los perfiles que lo tenían, y la marca `retirado:` impide que un reporte tardío lo recree.
5. **Si Redis pierde los datos:** al pedir "continuar viendo" de un perfil sin datos, se reconstruye desde `historial_por_perfil` (mes actual y, si está vacío, el anterior): la última visualización de cada contenido, con la misma regla que P4. La respuesta indica `origen: 'cassandra'`.

> **Por qué no en Neo4j ni en Mongo.** El progreso se escribe cada pocos segundos por cada persona que mira y se lee por clave exacta. Redis hace eso en memoria, en microsegundos, y con operaciones atómicas que además renuevan la sesión. Un grafo o una base de documentos pagarían un costo de escritura en disco sin ganar nada, porque nunca se recorre ni se busca por contenido.

11. Glosario

Término	Significado
ACID	Garantías de una transacción: atomicidad, consistencia, aislamiento, durabilidad
ALLOW FILTERING	Permiso de Cassandra para filtrar sin la clave de partición; recorre todo el cluster. No se usa en el proyecto
Atomicidad	Que un grupo de operaciones se haga entero o nada
BASE	Disponible casi siempre, estado blando, consistencia eventual: la filosofía de muchas NoSQL
CAP	Ante una partición de red, un sistema distribuido elige entre consistencia y disponibilidad
Clave de partición	En Cassandra, define en qué nodo vive una fila y agrupa las filas que se leen juntas
Clave de clustering	En Cassandra, ordena las filas dentro de una partición
Compensación	Acción que deshace un paso de una saga
Consistencia eventual	Las réplicas pueden diferir un rato, pero terminan coincidiendo
Cypher	Lenguaje de consultas de Neo4j, basado en patrones de nodos y relaciones
DATERANGE	Tipo de PostgreSQL para un intervalo de fechas
Desnormalizar	Copiar datos para no tener que juntarlos al leer
EXCLUDE	Restricción de PostgreSQL que prohíbe filas que cumplan todas las comparaciones a la vez (por ejemplo, superposición)
Heartbeat	Señal periódica de "sigo vivo"; acá, cada reporte de progreso renueva la sesión
Idempotencia	Repetir una operación deja el mismo resultado que hacerla una vez
Índice	Estructura que permite encontrar datos sin recorrer todo
Lua (script)	Código que Redis ejecuta de forma atómica
MERGE	En Cypher, crear un patrón solo si no existe
Normalizar	Guardar cada dato una sola vez, repartido en tablas relacionadas
Persistencia políglota	Usar varios motores, cada uno para los datos que mejor maneja
Pipeline de agregación	En Mongo, secuencia de etapas que transforma documentos
Saga	Operación dividida en pasos con compensaciones, para coordinar varios motores sin una transacción común
Score	El número que ordena un sorted set de Redis
TTL	Tiempo de vida de una clave en Redis; al vencer, se borra sola
Upsert	Actualizar si existe, insertar si no
UUID	Identificador único universal; acá, el id compartido entre los cinco motores